52/55

Homework 3

William Flanders*
Haverford College
(Dated: February 16, 2023)

1. NEWMAN EXERCISE 3.6

Nice explanation of context!

We want to investigate deterministic chaos particularly with Feigenbaum plots. A chaotic system can be best described as a system where the outputs are seemingly wild and random. With deterministic chaos, the output actually isn't random at all. A deterministic chaotic system has end results that actually are predictable, but often these results are very sensitive to change in input, and can vary wildly. For example, the weather is a deterministic chaotic system. If a being, say a demon, had information of every particle and molecule in the atmosphere today, it could theoretically predict how the weather would turn out tomorrow. Weather is chaotic because a small change in moisture or temperature or any other input variable can lead to two very different predictions.

The chaotic system we look to study is the logistic map

$$x' = rx(1-x) \tag{1}$$

where r is a parameter, x is the input, and x' is the output. What's remarkable about the logistic map is that it is a iterative recursive equation. The output x' becomes the input for the next iteration. This system has three long term behaviors dependent on the input and value of r. First, the system can settle down to a fixed value. For example, when $x = 0$, the system will remain at $x = 0$ regardless of the value for r. The system can also exhibit a limit cycle. This is where the system experience long term periodic behavior, cycling through the same values. Finally, the system can be truly chaotic, where the system will output a sequence of numbers that appear to have no pattern.

We can build an algorithm in python that showcases this system's behavior for a range of values of r. We can then plot values of x for corresponding values of r for a set number of iterations (in this case 1000). We can plot r vs. x to form a Feigenbaum plot. These plots are remarkable for their tree-like shape. We start with pseudo code

```
import matplotlib

initialize variables

# We can go through one full iteration
# before using the algorithm to ensure
# we will be analyzing long term behavior

for i in range(1000):
    x' = rx(1-x)

# We will want to iterate across all values
# of r from 0 to 4 with a step size of 0.01

while r < 4:
    r = r + 0.01
    for i in range(1000):
        x' = rx(1-x)
        save to lists

plot(r, x)
```

Figure 1 shows the end result after we run the algorithm. We chose 1000 iterations for the interval of r from 1 to 4 with a step size of 0.01. Our initial value for x was 0.5.

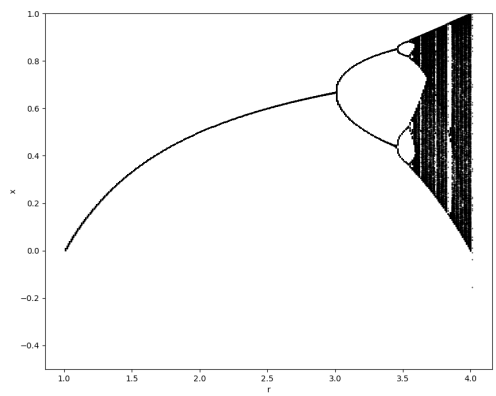

FIG. 1: Feigenbaum plot for a logistic map.

The shape of the plot looks like a felled tree, where we can see the trunk split off into two branches at just around $r = 3$. These two branches split again at just before $r = 3.5$ and then we begin to see the chaotic behavior manifest (what is sometimes called the edge of chaos).

For any given value of r, on a Feigenbaum plot, a fixed value would look like a straight horizontal line. No matter what value we choose for r, the logistic map will always output the same value of x across all iterations. We can see limit cycle behavior emerge when the single curve splits into two. These two split curves then split into two

*Electronic address: wflanders@haverford.edu; URL:

curves again netting a total of four curves, and from here we can see how the periodic nature of the logistic map breaks out for this interval of r. Finally, chaos behavior emerges where the plot no longer has structure or order and is seemingly just a collection of a bunch of random points. This area almost looks shaded and appears after the limit cycle interval.

2. NEWMAN EXERCISE 3.7

We want to investigate fractals through the Mandelbrot set. Fractals are mathematical structures that can infinitely be scaled in or out and still retain their characteristic shape. One such fractal forms from the Mandelbrot set. The set, which is exists in the complex plane, can be defined by

For a given complex value of c, start with $z = 0$ and iterate repeatedly. If the magnitude $|z|$ of the resulting value is ever greater than 2, then the point in the complex plane at position c is not in the Mandelbrot set, otherwise it is in the set.

Within the complex plane, we can treat x values as real where $-2 \leq x \leq 2$ and y values as imaginary where $-2 \leq y \leq 2$. We can build an empty N x N grid, check every point within the grid to see if it is an element of the Mandelbrot set, and then build out a plot accordingly. A true Mandelbrot set holds for an infinite number of iterations and for all values of x and y within the intervals. For our code however, since this isn't computationally feasible, we will build out our intervals with a step size of roughly 1000 (and a grid with roughly 1000 points) and check the magnitude of $|z|$ within 100 iterations.

We start with pseudo code

```
import matplotlib
import numpy

x = interval from -2 to 2 with 1000 steps
y = interval from -2 to 2 with 1000 steps
grid = empty grid of 0's that is N x N wide


loop through all values of x:
    loop through all values of y:
        initialize z
        turn x and y into a complex number
        iterate 100 times:
            check_z = the magnitude of |z| +
                      our craeted complex
                      number
            if check_z > 2:
                grid[x coord, y coord] = 1
                break the iteration
Create the plot
where 0's are black and 1's are white

plt.show()
```

Figure 2 is the plot for our fractal with the given parameters.

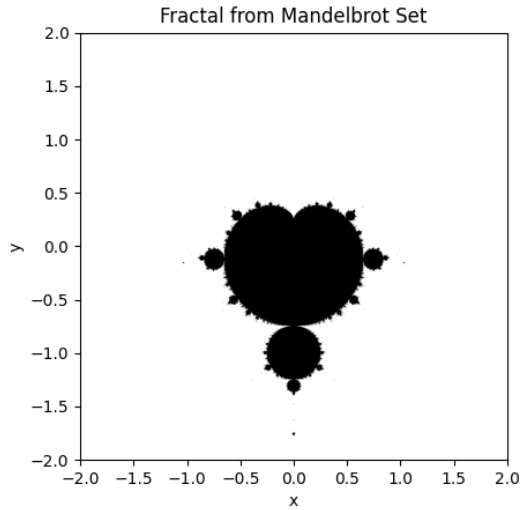

FIG. 2: Plot of fractal formed from the Mandelbrot set.

The fractal's shape is very unique and almost difficult to describe (nonetheless beautiful).

3. SURVEY

The problem set took roughly 5-6 hours. This week, the latex was much more manageable. The difficulty in python programming, however, definitely stepped up a level. Especially for the Mandelbrot set exercise, I often found myself researching complex numbers in python and brainstorming how I would come up with the plot.

Great write up!

3.6

Computational Physics/Astrophysics, Winter 2023:

Grading Rubrics [1]

Haverford College, Prof. Daniel Grin

For coding assignments, roughly 25 points will be available per problem.

1. Does the program complete without crashing in a reasonable time frame? If yes, up to +3 points. 3

2. Does the program use the exact program files given (if given), and produce an answer in the specified format? If yes, +1 points 1

3. Does the code follow the problem specifications (i.e numerical method; output requested etc.) Up to +2 points 2

4. Is the answer correct? Up to+4points 4

5. Is the code readable? Up to+2points 2

. 5.1. Are variables named reasonably?

. 5.2. Are the user-functions and imports used?

. 5.3. Are units explained (if necessary)?

. 5.4. Are algorithms found on the internet/book/etc. properly attributed?

[1] Inspired by rubric of D. Narayanan, U. Florida, and C. Cooksey, U. Hawaii

6. Is the code well documented? +3points 2

. 6.1. Is the code author named? -1

. 6.2. Are the functions described and ambiguous variables defined?

. 6.3. Is the code functionality (i.e. can I run it easily enough?) documented?

7. LaTeX writeup (up to 10 points) 9

- Are key figures and numbers from the problem given? (3 points)
- Is a brief explanation of physical context given? (2 points)
- If relevant, are helpful analytic scalings or known solutions given? (1 point)
- Are 3-4 key equations listed (preferably the ones solved in the programming assignment) and algorithms named? (2 points)
- Are collaborators clearly acknowledged? (1 point)
- Are any outside references appropriately cited? (1 point) Plot is missing labels -1 Otherwise great!

Note, even if (1), (2), (3), or (4) are not correct, one can still obtain many points via (5), (6), and (7).

$\frac{23}{25}$

2.7

Computational Physics/Astrophysics, Winter 2023:

Grading Rubrics [1]

Haverford College, Prof. Daniel Grin

For coding assignments, roughly 25 points will be available per problem.

1. Does the program complete without crashing in a reasonable time frame? If yes, up to +3 points. 3

2. Does the program use the exact program files given (if given), and produce an answer in the specified format? If yes, +1 points 1

3. Does the code follow the problem specifications (i.e numerical method; output requested etc.) Up to +2 points 2

4. Is the answer correct? Up to+4points 4

5. Is the code readable? Up to+2points 2

. 5.1. Are variables named reasonably?

. 5.2. Are the user-functions and imports used?

. 5.3. Are units explained (if necessary)?

. 5.4. Are algorithms found on the internet/book/etc. properly attributed?

[1] Inspired by rubric of D. Narayanan, U. Florida, and C. Cooksey, U. Hawaii

6. Is the code well documented? +3points 3

. 6.1. Is the code author named? -1

. 6.2. Are the functions described and ambiguous variables defined?

. 6.3. Is the code functionality (i.e. can I run it easily enough?) documented?

7. LaTeX writeup (up to 10 points) 10

- Are key figures and numbers from the problem given? (3 points)
- Is a brief explanation of physical context given? (2 points)
- If relevant, are helpful analytic scalings or known solutions given? (1 point)
- Are 3-4 key equations listed (preferably the ones solved in the programming assignment) and algorithms named? (2 points)
- Are collaborators clearly acknowledged? (1 point)
- Are any outside references appropriately cited? (1 point)

Note, even if (1), (2), (3), or (4) are not correct, one can still obtain many points via (5), (6), and (7).

$\frac{24}{25}$